

Rough draft regarding Tensor Decomposition

Written by:
Tymon Hendershott & Thomas Kidane

Wednesday, March 26, 2025

1. Inciting Problem
2. Notation
3. Canonical Polyadic Decomposition (CPD)
4. Tucker Decomposition
5. Applications

INCITING PROBLEM

Tensor decomposition comes up in many different problems. To ground our investigation in a concrete example, consider a black-and-white video. Suppose we want to encode the video's information in as little information as possible. Intuitively, we understand that certain

The structure of the video itself can be viewed as a three-dimensional tensor: each frame has a 2D grid of pixels, and each pixel has a certain luminosity (it follows that color would be a 4D tensor, but for pedagogical clarity we'll look at the simpler black-and-white case), which can be represented as some number between 0 and 1. It naturally follows that there may be redundant information across frames or parts of each frame, giving way to the ability to reduce the amount of information needed to recreate the original video, and motivating tensor decomposition. With this, we can TENSOR RANK

Tensor rank is the minimum number of rank-1 vectors needed to produce as their sum.

A key property of tensors is their uniqueness. Which comes from the fact that a decomposition has to fulfill more stringent requirements than from the fact that each layer of the tensor is itself another tensor, therefore there is a strong relationship between the layers of the tensor making it so that any rank one vector in some decomposition has to work for every slice of the tensor.

By restricting ourselves to low-rank approximations of the matrix, we get to understand the latent structure of the relationships.

Below, we have different generalizations of SVD to tensors, but it is important to note that there is no single generalization. Rather, depending on the purpose, certain decompositions are preferable over others. For example, it is advised to use CPD for latent parameter estimation and Tucker for subspace estimation, compression, and dimensionality reduction.

NOTATION

The concept of the outer product is defined as $X = a \otimes b = ab^T$, producing the matrix X . This can be viewed as a two-dimensional version of the general tensor product:

$$X = a^{(1)} \otimes a^{(2)} \otimes \dots \otimes a^{(n)} \text{ with } x_{i_1 i_2 \dots i_n} = a_{i_1}^{(1)} a_{i_2}^{(2)} \dots a_{i_n}^{(n)}$$

The rank of a tensor $\mathcal{X}$ is the minimum number of rank-one tensors needed to produce $\mathcal{X}$ as its sum. For a general rank- n tensor, this takes the following form:

$$\mathcal{X} = \sum_{i=1}^n \lambda_n a_i^{(1)} \otimes \dots \otimes a_i^{(n)} = [[\lambda; A^{(1)}, \dots, A^{(n)}]]$$

Where λ contains all the coefficients.

There are different ways of indexing a tensor, one way which we will use extensively in this paper is by fixing some of the indexes of the tensor and varying the others, creating different subarrays and fields of the tensor. In the simplest case we have fibers, which we get by fixing all but one index, slabs are made by fixing all but 2 indexes.

We define the order of tensor as the number of its dimension, so an order 2 tensor is just a matrix and an order 1 tensor is just a vector. For a 3rd order tensor, we can index its fibers as $\mathcal{X}_{:jk}$ (columns), $\mathcal{X}_{i:k}$ (row) and $\mathcal{X}_{ij:}$ (tube). In general we refer to the n -mode fiber of a tensor by fixing all but the n^{th} index.

0.1 Tensor Reorderings

We can turn a tensor into a matrix, and we will use this technique extensively in our algorithms. There are many of reordering a tensor into a matrix, but we will only look at the mode- n matricization.

The mode n matricization of a tensor turns $\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$, denoted $\mathbf{X}_{(n)} \in \mathbb{R}^{\mathbf{I}_n \cdots \mathbf{I}_{n-1} \cdot \mathbf{I}_{n+1} \cdots \mathbf{I}_N}$, turns the mode- n fibers into columns.

CANONICAL POLYADIC DECOMPOSITION (CPD)

The first type of tensor decomposition is rank decomposition, which is essentially expressing a tensor as a finite sum of rank 1 tensors. The most prominent decompositions are canonical decomposition position(CANDECOMP) and parallel factors(PARAFAC) decomposition. These two methods boil down to the same principles, therefore we will refer to them as just CPD.

The 3-way CPD case is as follows.

$$\min_{\hat{X}} \text{ where } \hat{X} = \sum_{r=1}^R a_r \otimes b_r \otimes c_r = [[A, B, C]]$$

If they are the same $\hat{X}$ is an exact low rank approximation to X. Which is the same as:

$$\begin{aligned}\hat{X}_{(1)} &= (C \odot B)A^T \\ \hat{X}_{(2)} &= (C \odot A)B^T \\ \hat{X}_{(3)} &= (B \odot A)C^T\end{aligned}$$

For the general case we have

$$\hat{X} = \sum_{r=1}^R \lambda_r a_r^{(1)} \otimes a_r^{(2)} \otimes \dots \otimes a_r^{(n)} = [[\lambda; A^{(1)}, A^{(2)}, \dots, A^{(n)}]]$$

$$\hat{X}_{(n)} = \Lambda(A^{(N)} \odot \dots \odot A^{(n+1)} \odot A^{(N-1)} \odot \dots \odot A^{(1)})A^{(n)T}$$

Where $\Lambda = \text{Diag}(\lambda)$

We will look at 1 CPD algorithm called the Alternating Least Squares Algorithm

0.2 Alternating Least Squares(ALS)

This decomposition is the most widely used type of decomposition out in the real world, and the key idea behind it is just to iteratively fix one matrix and apply least squares algorithm(which is very well understood) to decrease the error while holding all the other matrices fixed, and do this as many times until some halting condition is reached. This algorithm works for a fixed rank, so you have to set the required rank of the tensor to use the algorithm.

In the case of a 3-way tensor, the decomposition would repeat the below steps.

$$\begin{aligned}A &\leftarrow \arg \min_A ||X_{(1)} - (C \otimes B)A^T|| \\ B &\leftarrow \arg \min_B ||X_{(2)} - (C \otimes B)A^T|| \\ C &\leftarrow \arg \min_C ||X_{(3)} - (C \otimes B)A^T||\end{aligned}$$

Now if you squint your eyes at the equation you will see why it is called alternating least squares, since the solution to these problems is the same as solving the least squares problem for one case and then alternatingly solving it for the other ones.

0.2.1 Developing the Algorithm

Well first step would be to define what our inputs would. Since are trying to approximate the tensor, we first of all need what we are going to approximate, denote the tensor with $\mathcal{X}$.

```
procedure ALS( $\mathcal{X}$ )
  dosomething
```

We also need the rank R . But here lies a question, what should R be? Without seeing the tensor at all, there is nothing much we can say about what the rank should be, but it is computationally expensive to try to optimize a large number of matrices, so in the event that our algorithm doesn't have a set rank, we should start at 1, and progressively increase it, until we get the desired result. But another question is when do we decide to increase the rank? This question is very situation dependent, so we will include a small section that checks this condition for increasing the rank.

```
procedure ALS( $\mathcal{X}$ ,  $R=1$ )
  dosomething
```

Now we also need to initialize the set of matrices we are going to be working on.

```
procedure ALS( $\mathcal{X}$ ,  $R=1$ )
  if  $R$  not set then
    for  $i \leftarrow 1$  to  $\infty$  do
      ALS( $\mathcal{X}$ ,  $i$ )
  else
    do something
```

Next we need to define a loop that will iterate over the different matrices, and minimize the loss, for this we will do a mode n , matricization.

TUCKER DECOMPOSITION

To begin, let us review Singular Value Decomposition (SVD). Tucker Decomposition builds upon the generalized properties of SVD, extending them into higher dimensions. SVD decomposes a vector into 3 matrices:

```

procedure ALS( $\mathcal{X}, R=1$ )

```

```

    if  $R$  not set then

```

```

        for  $i \leftarrow 1$  to  $\infty$  do

```

```

            ALS( $\mathcal{X}, i$ )

```

```

    else

```

```

        initialize  $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R}$  for  $n = 1, \dots, N$ 

```

```

procedure ALS( $\mathcal{X}, R=1$ )

```

```

    if  $R$  not set then

```

```

        for  $i \leftarrow 1$  to  $\infty$  do

```

```

            ALS( $\mathcal{X}, i$ )

```

```

    else

```

```

        initialize  $\mathbf{A}^{(n)} \in \mathbb{R}^{I_n \times R}$  for  $n = 1, \dots, N$ 

```

```

        while not reached stopping condition do

```

```

            for  $n = 1, \dots, N$  do

```

```

                 $V \leftarrow A^{(1)T} A^{(1)} * \dots * A^{(n-1)T} A^{(n-1)} * A^{(n+1)} A^{(n+1)} * \dots * A^{(N)T} A^{(N)}$ 

```

```

                 $A^{(n)} \leftarrow \mathcal{X}_{(n)}(A^{(N)} \otimes \dots \otimes A^{(n+1)} \otimes A^{(n-1)} \otimes \dots \otimes A^{(1)}) V^\dagger$ 

```

- U , a (generally) singular matrix containing the orthonormal eigenvectors of AA^T ,
- Σ , a diagonal matrix of singular values, and
- V , a (generally) singular matrix containing the orthonormal eigenvectors of $A^T A$.

With 2-dimensional Tucker decomposition, or "matrix SVD", generally U and V are not of the forms $m \times 1$ (which would be a vector). This follows intuitively from the fact that most matrices cannot be represented as an outer product of two vectors. This point is emphasized to explain the idea behind the "rank" of decomposed tensors; it is usually not possible to represent a tensor as the decomposition of just one rank-1 tensor. Thus, the crux of Tucker decomposition lies in finding the smallest number of tensors to add together to get our original tensor (or, to approximate it to some controlled degree of error).

In n dimensions, Tucker decomposition manifests itself as n factor matrices and an n -dimensional hypercube tensor (see below image):

In the above figure, $\mathcal{G}$ is the core tensor, where $\mathcal{G} \in \mathbb{R}^{R_1 \times R_2 \times R_3}$ (note to self, R_2 is the purple one), and the factor matrices are:

$$U^{(1)} \in \mathbb{R}^{I \times R_1}, U^{(2)} \in \mathbb{R}^{J \times R_2}, U^{(3)} \in \mathbb{R}^{K \times R_3}$$

The core tensor $\mathcal{G}$ dnesly expresses the extent to which (and how) different tensor elements interact with each other. Meanwhile, the factor matrices represent the "parts we are compressing". The aim of Tucker decomposition is to capture latent multi-way relationships between variables while minimizing the memory allocation required.

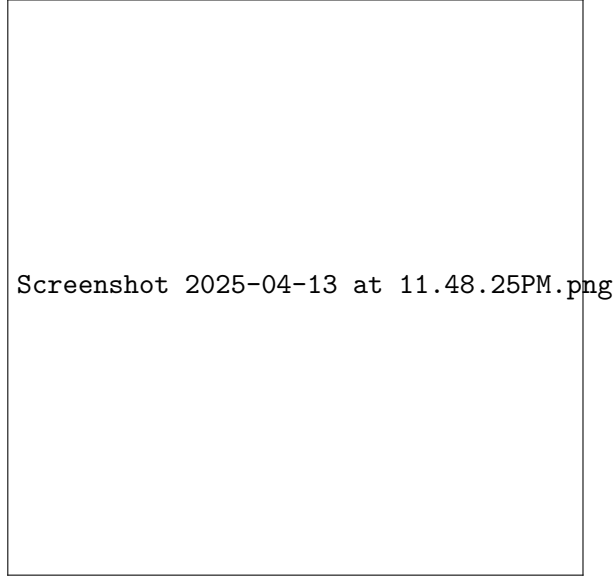


Figure 1: Tucker Decomposition (from Introduction to Tensor Decompositions and their Applications in Machine Learning, Rabanser et al)

Naturally, the question arises: where do R_1, R_2 , and R_3 come from? Depending upon our application, the truthful answer is that they depend. When tensors are small, and there are few dimensions (as in the diagram above) we can easily compute the decomposition in full. However, for far larger applications, in which tensors can encompass terabytes of data, and in which the dimension can be far larger than 2 or 3, it becomes necessary to approximate Tucker decomposition to some degree of error in order to be able to meaningfully compute it (more on this later).

For the above 3-dimensional case, Tucker decomposition looks like this:

$$\begin{aligned} \min_{\hat{\mathcal{X}}} \|\mathcal{X} - \hat{\mathcal{X}}\|, \text{ where } \hat{\mathcal{X}} &= \sum_{u_1=1}^{U^{(1)}} \sum_{u_2=1}^{U^{(2)}} \sum_{u_3=1}^{U^{(3)}} \{a_{u_1}^{(1)} \otimes a_{u_2}^{(2)} \otimes a_{u_3}^{(3)}\} \\ &= \mathcal{G} \times_1 A^{(1)} \times_2 A^{(2)} \times_3 A^{(3)} = [[A^{(1)}, A^{(2)}, A^{(3)}]] \end{aligned}$$

More generally, the n-case looks as follows:

$$\begin{aligned} \min_{\hat{\mathcal{X}}} \|\mathcal{X} - \hat{\mathcal{X}}\|, \text{ where } \hat{\mathcal{X}} &= \sum_{u_1=1}^{U^{(1)}} \cdots \sum_{u_N=1}^{U^{(N)}} \{a_{i_1 u_1}^{(1)} \otimes \cdots \otimes a_{i_N u_N}^{(N)}\} \\ &= \mathcal{G} \times_1 A^{(1)} \times_2 \cdots \times_N A^{(N)} = [[A^{(1)}, \dots, A^{(N)}]] \end{aligned}$$